Learning Easy Optimal Policies

Module: Research Project ELEC0054, 2025-26
Student number: 24249071

Abstract

Robustness in reinforcement learning is usually gained through training, by specifying an uncertainty set, randomising simulation parameters, and training against adversaries. Each of these methods requires fixing the axes for where the policy tolerates the error. This work instead tries to treat tolerance to error as a property of the state rather than policy or environment. I define the forgiveness of a state as the flatness of its action-value landscape, instantiated either as an action gap or the normalised entropy of a Boltzmann distribution over the searched values, and extend it to a visit-weighted aggregate over the subtree to account for turn-to-turn forgiveness variance. The statistic is estimated within an AlphaZero-style search in chess, with Gumbel top-m sampling and sequential halving at the root to make sibling estimates comparable, and it is predicted by a detached network head. I evaluate the robustness by injecting action mis-execution at a rate ϵ and measuring the resulting loss in Elo. The head learns a non-trivial quantity, reaching $R^2 = 0.74$ against 0.3 for regressions of raw board features. Steering towards this forgiveness, however, did not improve the robustness of the agents: a δ tiebreak switches 40% of moves at a mean cost of 0.0045 in value while raising the average forgiveness by at most $0.004\breve{}0.007$, and loses 48.5 Elo at $\epsilon = 0.1$ against a matched control. I report the diagnosis and the conditions under which the hypothesis remains untested.

1 Introduction

Robustness is important in reinforcement learning because of differences in training and deployment of policies; there is often execution noise, environment shifts, or outside factors not accounted for. Models have to be made with this in mind; a model that is not robust to mistakes and differences will ultimately fail in the real world. Techniques generally achieve robustness through training, introducing uncertainties, randomised parameters, and adversaries, all requiring decisions for which disturbances matter in order to expose the model to them. This robustness only covers the axes chosen. These are enforcing tolerance to mistakes as properties of the policy or the training distribution. I should also be looking at tolerance to mistakes as a property of the current state; some states are naturally more forgiving than others and allow the model for more mistakes without them being as catastrophic. Two different policies with equal returns can have different amounts of time in different states, but one can be more robust to mistakes than the other, yet they won't be distinguished.

I can imagine the idea of "brittle" states where only a single action can be taken; if I take the wrong action, a second-best, or some form of mistake, this action leads to a far worse position. These are states where one action is far better than the others.

This brings up the idea of "forgiveness" in a state where a state is not "brittle" and multiple moves have similar Q-values, where the second-best move or mistaken move doesn't lead to as bad a position. States with multiple near-best actions. Within a state $V(s)$ is a fixed offset, $A(s, a) = Q(s, a) - V(s)$. Since the statistic is using within-state values, I can think about the Q-values instead of Advantage.

After defining an idea of local forgiveness, I can extend it as a recursive weighted quantity. Here, the forgiveness of a state is made up of the local forgiveness as well as the forgiveness of the states that actions in that state lead to, which is weighted by our policy to ground the lookahead forgiveness as realistic. A

state's forgiveness should account for what its actions lead to so that a locally safe move into a brittle sequence is not counted as safe.

In this paper, I have used chess as the environment and based the model on AlphaZero. The formulation is easy to visualise in chess, where a multi-move sacrificial line may be viewed as brittle, while having many good choices can be viewed as safe. Positions can vary enormously, and there is a clear idea of what a mistake is. The model has branching trees from MCTS that produce value estimates per state for each action and make the statistic easy to calculate. Chess is a well-optimised game, which can easily be generated cheaply for training and has a lot of optimisation existing already.

Using the calculated forgiveness value, I will explore changing the policy to seek forgiveness in tie-breaking near-equal actions, as well as shaping the search using the value. To test the robustness, I will perturb some of the agents' moves, and these can be compared across models to see how much strength is lost under these conditions.

2 Literature survey

2.1 Formulations through training

Classically, robustness is treated as a game against nature. Iyengar (2005) and Nilim & El Ghaoui (2005) replace the transition probabilities of each state-action pair with a set of plausible alternatives and define the value of a policy as the return under the worst choice of that set. The uncertainty set must be set up in advance, and resulting policies are frequently conservative since, in nature, the dynamics are not varied independently at each state, which is an assumption in this setup.

A widely adopted alternative to handle the sim-to-real gap, where the simulator doesn't accurately reflect the hardware even if it's only marginally off, is domain randomisation. Tobin et al. (2017) sample parameters from a distribution during training, where friction, textures, masses, etc., are all varied. A policy that learns to perform well across all of these distributions will perform well in the real environment where the exact parameters may differ. Rajeswaran et al. (2017) improves on this by concentrating learning on the worst-performing models to prevent policies that perform excellently on some models but poorly on others from having equal footing with policies that perform well on all models. The robustness obtained here is only on the same axes of variation that are given in training; the range of variation reduces strength. Too wide makes the model too cautious, but too narrow risks the real environment falling outside.

Building on formulations of RL as a game against a disturbance, Pinto et al. (2017) train an adversary that applies destabilising forces to the system while the policy has to maximise return. In this setup, specifying the perturbations by hand is not needed, at the cost of a second set of networks. The adversary still has to have a selected method of interference, so it learns how to disrupt, but the tools still need to be supplied.

2.1.1 Action-robust formulations

Perturbations can also be applied to the agent rather than environment, either in the input, where the observation is corrupted and not the state (Zhang et al., 2020), or on the output side, where the executed action is. The latter is the failure studied here. In Probabilistic Action Robust MDPs (PR-MDPs) (Tessler et al., 2019), the joint policy is defined with a mixed policy of an agent and an adversary, where the agent's policy is chosen with $1 - \alpha$ probability and the adversary's with α probability. The value of a policy is then defined as the discounted return of this mixed process minimised over all adversary policies. The same paper also defines a Noisy Action Robust MDP (NR-MDP), which moves the perturbation from the policy to the action itself.

These formulations are theoretical, but the vulnerability is measured: offline RL for robot control degrades sharply under perturbations injected into joint torque signals (Ayabe et al., 2025), the continuous-control analogue of the mis-execution model of § 5.3.

2.1.2 Maximum Entropy

All these approaches try to achieve robustness by changing what the agent trains on, through sampled environments or a learned adversary. Another method is to alter what the agent optimises. The MaxEnt method adds a bonus proportional to the entropy of the policy's action distribution (Ziebart et al., 2008; Haarnoja et al., 2018). Eysenbach & Levine (2022) show that the maximum-entropy objective maximises a lower bound on a robust RL objective and characterises the set of disturbances against which the policies are guaranteed to perform well. The optimal policy is Boltzmann, $\pi^*(a|s) \propto \exp(Q(s,a)/\alpha)$, with the temperature α controlling how greedy the policy is. The preference for a flat landscape and the spread of the executed policy are governed by the same temperature, and the robustness guarantee is coupled with the stochastic policy. MaxEnt does this without an explicit representation of which states are safer; the same temperature is applied all over, and the policy is made less decisive whether or not it is necessary. The present work follows the max entropy route but treats the structure within a state as a quantity to be estimated and targeted directly, instead of a by-product of a policy.

2.2 The action gap

The action gap is defined as the difference between the first- and second-best actions in the action space: $Q^*(s,a_1) - Q^*(s,a_2)$. Farahmand (2011) introduced the action-gap regularity as a property of the value function, parameterised by $\zeta \geq 0$ measuring how often actions are near tied. Because a greedy policy only depends on the value estimates, if the estimation error is smaller than the local gap, the action is unchanged. The performance loss of the greedy policy w.r.t $\hat{Q}$ bounded by $O(\|\hat{Q} - Q^*\|_\infty^{1+\zeta})$, decays faster than the estimation error. This explains how often agents attain near-optimal returns before the value estimates converge. It is therefore suggested that widening the action gap deliberately reduces the impact of estimation errors. Bellemare et al. (2016) introduce the Bellman operator, which changes the backup so that the values of suboptimal actions are lowered with the greedy action left intact, increasing the action gap at every state. The operator preserves the optimality: the greedy policy it induces is exactly the same, and the optimal value at any state is not changed. The extra margin of error is therefore obtained without cost and translates to a reduced sensitivity to estimation error. The literature, therefore, offers a comprehensive account for why states with a clear decision are desired. The argument for a wider gap is made entirely with respect to estimation error in the value estimate, but there are areas where I want the action gap to be reduced.

2.2.1 Different error models

The argument for widening the gap is based on potential errors in the value estimate, where the agent's $\hat{Q}$ differ from Q^*, and if the difference is enough to change the argmax. In this case, the gap is used as a safety margin, allowing the estimation error to be absorbed without changing the action. The action-robust criteria in § 2.1.1 describes a different failure type. I are looking at a misstep, where the agents' value estimates are correct with correct rankings, but the error occurs in execution, where the desired action is not carried out. The question is then what is lost once an action other than the argmax is played? The action gap in this case measures this cost directly, but with the reversed sign. A state where one action is far superior to the rest is the exact state where being forced off this action costs the most. If several actions are of near-equal value, or the action gap is small, the change in action costs almost nothing. The source of the error then changes what I are looking for. The two effects are not independent. A wide gap increases the cost of mistakes but also lowers the probability of making such mistakes. The two are therefore competing, and which dominates depends on the source and magnitude of error. Finding where the balance lies for a given perturbation can be found by holding policy fixed and varying the rule to select the action.

2.2.2 Flatness as an objective

In supervised learning, with parameterisation that has achieved a similar training loss, the ones in flatter regions have been argued to generalise better (Hochreiter & Schmidhuber, 1997; Keskar et al., 2017). Foret et al. (2021) make this an explicit objective rather than a by-product of the optimisation. The quantity is different from the one here, the flatness of a loss surface in parameter space against flatness of the action

values in a fixed state, the precedent making an observed property an explicit target makes the trade-off controllable.

2.3 Sibling comparability in search

The statistic proposed compares Q estimates across siblings, requiring them to be similarly reliable, which PUCT does not supply. Several lines of work address this problem. H-MCTS applies SHOT recursively near the root and reverts to UCT deeper (Pepels et al., 2014; Cazenave, 2015), Anytime SH approximates the schedule without a fixed budget (Sagers et al., 2025), and RMCTS instead solves the regularised posterior bottom-up (Frankston & Howard, 2026). MiniZero evaluates AlphaZero, MuZero and their Gumbel variants under a common framework on board and Atari games (Wu et al., 2024). These are alternative routes to comparing sibling forgiveness estimates, none of them measures the shape of the value landscape at a state.

3 Description of models

The platform for this work is the AlphaZero family (Silver et al., 2018; 2017), where I have a network supplying a prior policy and a value estimate to guide a Monte-Carlo tree search; the improved policy from the search goes back into the network.

3.1 Search and Evaluation

The network $f_\theta(s) = (p, v)$ maps a position to a prior distribution p over all the legal actions and a value $v \in [-1, 1]$ from the mover's POV. Each move is chosen after a search of n iterations. An iteration descends from the root node, and at each subsequent node, it selects the action maximising the PUCT score (Rosin, 2011)

$$a = \arg\max_a \left[Q(s,a) + c \cdot P(s,a) \cdot \frac{\sqrt{\sum_b N(s,b)}}{1 + N(s,a)} \right]$$

until it reaches an unexpanded or terminal node. The sign is alternated so that the nodes only accumulate the value from the POV of the player choosing. The action value is then the average $Q(s,a) = \frac{V(s,a)}{N(s,a)}$, with unvisited children taking a first-play-urgency reduction (Appendix E).

The PUCT rule is normally presented as a heuristic, but Grill et al. (2020) shows that it behaves like to an optimiser. The visit distribution tends to the solution of a regularised policy optimisation problem, so the action values are the results of the search and the visit counts are a summary of them which is noisy at lower budgets.

Three things are needed that PUCT search doesn't supply: **Comparable Q estimates**, both statistics are functions of the differences between Q values, so the estimates have to be similarly reliable across actions. PUCT focuses the search effort on a few nodes and selection is determined by the visit count, this does not matter for PUCT; however, it renders Q estimates for less-visited nodes noisy, nodes further down the tree are worse since the branching spreads the budget thinner. **Values for unvisited actions**, an unvisited action has no value, and if I just drop these values, this artificially increases the forgiveness stat after normalisation. **Policy target independent of allocation**, spreading out the visits to reduce noise on less optimal actions degrades the visit count target, switching to using the Q values removes this conflict.

3.2 Better Defining Q-values with Gumbel

One method that obtains better Q-value estimates is policy improvement through planning with Gumbel. Three root-level mechanisms are then replaced together, following Danihelka et al. (2022).

Top-m sampling AlphaZero explores the root node by adding Dirichlet noise to the priors, which changes the quantities that the search reasons with. Gumbel sampling moves the randomness into independent

$g(a) \sim \text{Gumbel}(0, 1)$ variables, which are added to the prior logits, and the m greatest values are taken. The priors are left as the network produced, and exploration is gained by the m actions considered.

Sequential Halving Sequential halving, Karnin et al. (2013), treats the root as a best-arm identification problem rather than the cumulative-regret problem that PUCT descends from (Kocsis & Szepesvári, 2006). The budget is split into $\lceil \log_2 m \rceil$ phases, each of which every surviving candidate equally visits and discards the worse half. The estimates compared at each elimination are matched by construction.

Halving terminates at two candidates, which is too narrow for the entropy forgiveness statistic. Normalised entropy over $n = 2$ is essentially the same as the action gap it was introduced to improve upon. The top four candidates are therefore also recorded at the final phase transition, where all four nodes still have visits of 87 or more; a floor was not applied here due to subtree reusing, a child can arrive with inherited visits and clear the floor without being a candidate. Interior nodes are left under plain PUCT, as the root is where both the target and the played move originate.

Replacing PUCT at the root changes the search relative to a standard AlphaZero agent, and results here are therefore not comparable to published AlphaZero figures. Every comparison reported is between two arms running the identical root procedure at the identical budget, so the halving schedule is common to both sides of every experiment and cannot account for a difference between them.

Policy target Using halving, the root visit counts become unusable as the target after each wave has the same visit counts, with $N(a)$ recording the elimination schedule rather than the quality of a. The replacement is $\pi'(a) = softmax\big(logit(a) + \sigma(\hat{q}(a))\big)$ Taken over all children of the root, σ scales the values onto the logit scale in proportion to how much of the budget has been spent. The Gumbel variables do not appear in this; they are resampled each move, whereas a target must be a deterministic function of the search. Calibration of σ is treated in Section 6. The same score selects which move is played, ranking candidates by prior-regularised values rather than visit counts, which is what allows a preference between two near-optimal paths to be expressed at all.

4 Forgiveness value

One way to define our local forgiveness $F_{\text{local}}(s)$ is with the size of the action gap, $G(s) = Q_1 - Q_2$, where the Q values are the action values in descending order.

The action gap only sees the top two actions, so in states with many moves, it can't separate advantage distributions differing substantially below runner-up. Consider a state whose actions have values 0.8, 0.79, 0.1, and 0.1, and a second state with values 0.8, 0.75, 0.75, and 0.7. The first has a smaller action gap; however, the second offers more viable actions and would arguably be more forgiving. A statistic that's sensitive to the whole near-optimal set is wanted, so local forgiveness is defined as the normalised Shannon entropy of a Boltzmann distribution over the searched values at the node. The Boltzmann distribution is as follows:

$$p(a) = \frac{\exp Q(a)/\tau}{\sum_b \exp Q(b)/\tau}, \qquad H(p) = -\sum_a p_a \log p_a, \qquad F_{\text{local}}(s) = \frac{H(p)}{\log n}. \tag{1}$$

With n being the number of qualified actions, as in § 3.2, and $k_{eff} = \exp H(p)$ the rough near-optimal actions. The normalisation renders F the fraction of the available choices that are usable, so the values are comparable among different positions with differing numbers of legal moves.

4.1 Calculating Downline Forgiveness

A states forgiveness should take into account the states its actions lead to, so that a locally safe move into a brittle state is not counted as safe and vice versa. With $D(s)$ the downstream subtree of a state, including the current state I can define

$$F(s) = \sum_{s' \in D(s)} F_{\text{local}}(s') \cdot \frac{N(s')}{N(D(s))}$$

with $N(D(s))$ being the total sum of visits over the subtree. In this formulation, nodes closer to state s receive more weight because, as the tree branches, the further-along states receive fewer visits as a consequence of search. A recursive form, discounting each ply by γ rather than weighting in one pass was implemented, but the two correlate at $0.97 - 0.99$ (Table 4) and only the flat aggregate is used. In both of these forms, these can only be calculated once search is complete so they can inform selection but not the search; § 4.2 introduces the head to lift this restriction.

Nothing in the entropy regularisation or action gap literature distinguishes whose slack is accumulated. There are a few options for which nodes from the tree to count, since at every node, both sides of the board are calculated, so the aggregate can count all nodes, only the mover's, only the opponents, or the mover's minus the opponents at decay γ. The second is what the tiebreak approximates, and the fourth $F_{me} - \gamma F_{opp}$, is what the alternating sign backup from § 4.3 implements.

4.2 Forgiveness Head

The forgiveness head is required for leaf nodes that have no children and nodes with few visits and noisy Q values. The head is detached, so its gradient never reaches the trunk. Swapping one head for another under the tiebreak selection rule cannot alter the policy or value outputs and therefore cannot alter the search. Two arms differing only in which head they load are the same player under two selection rules, which is what makes the parity comparison in § 6.2 an isolated test of the statistic rather than of two different networks.

The head is evaluated at the position currently being looked at, selecting for our own forgiveness then requires using the opponent's forgiveness _opp head. When shaping, the mover's forgiveness is used on the leaves, since each leaf is from the mover's POV the mover's forgiveness _me is used and alternates sign for the opposing leaves.

Training can only be completed at the root nodes, where the search iterations are sufficient. The signal used to train the head will be the flat tree forgiveness value so that for any child of a node, the rough down-line forgiveness can be calculated.

4.3 Shaping

Since I want to seek out more forgiving states without a loss of strength, I can use a tie-breaker within δ of the best move. This only switches a move if there are two or more equally good moves where one has a greater forgiveness value.

Another way to implement is to shape the search, modifying the value to include a forgiveness term that every lead backs up, $v' = \text{clip}\big(v + \beta(2\hat{F} - 1), -1, 1\big)$ where $\hat{F}$ the forgiveness head's output is in the point of view of the player to move. Both of these are approximations of the same objective. Taking z as the game outcome from the mover's POV, the baseline maximises $J(\pi) = \mathbb{E}_\pi[z]$, where I maximise

$$J_\beta(\pi) = \mathbb{E}_\pi\Big[\, z + \frac{\beta}{T} \sum_{t=0}^{T-1} \epsilon_t \big(2F(s_t) - 1\big)\Big], \qquad \epsilon_t = \begin{cases} +1 & \text{if the agent moves at } s_t \\ -1 & \text{otherwise} \end{cases}$$

Setting $\epsilon_t \equiv +1$ recovers the mover-only form, which is what the tiebreaker approximates, while the alternating ϵ_t is what the shaped search implements, since a leaf is evaluated in its own mover's POV and the backup flips sign per ply. The centring places $2F(s) - 1 \in [-1, 1]$ on the same scale as z. Averaging maintains the bonus per move so that β does not have to be returned as games become longer and $\beta = 0$ recovers the baseline. The δ tiebreak is the constrained form of this, maximising F subject to giving up no more than δ value per move, and the shaped value is the penalised form, paying for forgiveness at a fixed rate rather than within a fixed budget. Since the shift applied at a leaf is at most 2β, taking $\beta \approx \delta/2$ makes the two about as willing to pay as each other.

Both objectives add an entropy bonus, they just differ in the distributions. MaxEnt rewards $H(\pi(\cdot|s))$, the bonus is paid when the agent acts less decisively. Here the entropy is taken over the Boltzmann distribution on $Q(s,\cdot)$, which is not sampled from and describes the shape of the value landscape at s. The temperatures are then separate, with τ only setting what counts as near-optimal when measuring F, so unlike MaxEnt, the agent prefers flat states without having to act stochastically once it reaches them. The bonus is also deliberately not of the potential-difference form $\gamma\Phi(s') - \Phi(s)$ which is policy-invariant shaping, as in Ng et al. (1999); β is the rate at which the win probability is traded for slack.

5 Implementation

For the main training loop, I use a self-play RL loop, where the network plays games against itself, generating policy and value targets that it trains on, which subsequently improve over time. There is a buffer of 600,000 positions so that as play goes on, the targets are not biased towards the recently favoured moves and include a wider range of pointers. For the forgiveness heads, they train offline, using games created by the saved model. As the heads use the backbone of the network, it's optimal to train them offline to not pass moving inputs which don't get influenced by this target. This is also biased towards the sort of games the network produces, resulting in moves that the head is more likely to see. A fixed dataset is used from 512 generated games, which is sampled and trained on.

5.1 Environment, encoding and network

For the chess environment, bitboards are used with one 64-bit integer representing the pieces of one side and type. Cython is used for the move generations for maximum speed. Position encoding is used with 19 planes of 8x8, 12-piece planes in the mover's POV, 4 for castling rights, en passant, halfmove clock scaled by 100, and repetition count. This fully expresses a move, and since they are mover POV, they can represent either side instead of training both colours. Search scores repetitions, as in standard chess, and after 50 moves, requiring the value head to have these as input. The action space is 64x73 (4672), which represents every possible move, including underpromotions. PyTorch (Paszke et al., 2019), NumPy (Harris et al., 2020) and Cython (Behnel et al., 2011) were used here.

The network is built of 128 channels, 8 residual blocks, and a convolutional stem that takes the 19 input planes. The heads are the value head (scalar), policy head (4672 logits), and forgiveness (scalar), with the first two and the stem standard for AlphaZero but scaled down. The forgiveness head is detached so that it does not influence the stem to bias forgiveness. The network holds 3.08M parameters, of which the forgiveness head accounts for only 4,355, roughly 0.14%, since it reads a single detached 1-channel projection of the trunk.

5.2 Self-play generation

When generating the states in self-play to train on, there are 192 games played per training iteration, 800 search iterations per move, with only 50% of moves being played fully, and the fast moves only using 100 iterations. Fast moves only train the value head and skip the policy and have been shown to not reduce training while speeding it up Wu (2019). There is no data split for training, evaluation, and testing; the buffer replaces these.

Using batches of 256, with 64 batches trained per iteration, and approximately 16k rows consumed per iteration. For losses, I have used cross-entropy against the target distribution. For value, I have used MSE and forgiveness MSE, masked so that undefined rows produce nothing. Optimisation uses AdamW with a separate optimiser and schedule for the forgiveness heads (Appendix D).

5.3 Evaluation harness

Robustness is measured as strength under injected execution errors. At each post-opening decision the agent's selected move is replaced, with probability ϵ, by a different move from the halving finalists; the value estimates and the search are untouched, so this is the misstep failure described in § 2.2.1 rather than an

estimation error. Openings come from a book and are played unperturbed so that games diverge from a common set of starts. Realised perturbation rates are recorded per run and matched nominal ϵ to within 0.005 throughout.

Two pairings are used. With head to head, two arms play each other, with both being perturbed during play. In benchmark mode, each arm plays against a fixed third checkpoint, giving independent curves that can be compared. Runs play 200, 400 or 800 games per arm, per ϵ with Elo being reported with 95% confidence bounds.

Elo differences are obtained by inverting the logistic expectation on the mean per game score; they are given with 95% confidence intervals transformed through the same expression (Appendix C).

5.4 Hyperparameters and calibration

To calibrate τ, I probed 300 positions from the deployed checkpoint and took the median action gap, 0.0123, setting $\tau = \text{median}(G)/\ln 2 = 0.0178$ so that the median position maps to $F_{local} = 0.5$. The same τ is used for entropy, where it plays the same role, I cross-checked this and the median entropy lands on 0.5.

To calibrate δ, I searched 390 positions at the deployment budget of 800 simulations and again at 5000 to obtain a root mean square action gap disagreement of 0.031 among positions that have a gap of at most 0.1, so the positions could be acted upon. Setting $\delta = 0.05$ places it above the noise level and would perturb an action in 66% of the positions at a mean cost of at most 0.012 per move. This is actually measured at a switch of 39.5% in deployment, due to differences in the probe set and in game distribution.

To calibrate β, the requirement is that the penalised form pays for forgiveness at around the same rate as δ. The shift at a leaf is, at most, 2β, so $\beta = \delta/2 = 0.025$ matches. The shaped run uses $\beta = 0.03$, which pays at 1.2 times the rate of the δ tiebreak. It also sits below the 0.031 search noise, so no individual-shaped decision is resolved on the bonus alone. However, as it is systematic, it accumulates in one direction across positions.

To calibrate c_{scale}, I matched the entropy of the new target to that of the visit-count targets the policy head had been trained against, near 2 nats. With the rescaling active, $c_{scale} = 0.02$ yields roughly 2.1 nats, which is essentially equivalent to eight effective moves, 0.05 yields 0.7, and 0.10 yields 0.15. Therefore, the paper's 1.0 and mctx's 0.1 (Babuschkin et al., 2021) are both far too sharp at this budget; 0.02 changes the shape of the target without also changing its temperature.

The hyperparameter table is located in the appendix, along with the compute and reproducibility settings. (Table 3)

6 Experimental Results and Analysis

6.1 Validating the forgiveness target

The forgiveness head can only be used if it actually predicts something that cannot be seen in the raw encoding. It could, for example, only count how many pieces are left. Before deploying the heads, I compared each trained head against linear regressions predicting the same target from features read off the encoded planes without network and search: the piece counts, how quiet the position is, the material balance, and the per-type counts, added cumulatively. If a head matched these, it would have learned nothing beyond trivial signals.

Table 1 shows that the piece count accounts for a around $20 - 30\%$ of the variance in the lookahead targets, using fifteen features adds at most 6%. The signal is almost immediately saturated, so a larger feature set would not close the gap. Dependence on the amount of material is normal, the number of pieces can correlate both to the number of legal moves and to complexity for an endgame, where every move may be similar. Since I have a good margin on the deployed target, flat_entropy_me, with a 43% margin, there is a real statistic learnt here that I can use.

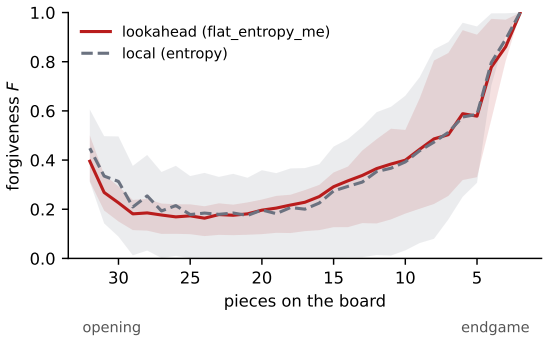


Figure 1: Forgiveness against material. Median F over decisions from 400 self-play games at the deployed checkpoint, the band is the interquartile range.

Table 1: Trivial-feature controls. OLS fits predicting each target from features read off the encoded planes; no network or search. Full ten-target table in Appendix 5. † deployed.

	Control R^2			
Target	pieces	all 15	**Head**	**Margin**
`entropy` (local)	0.133	0.190	0.602	+0.412
`flat_entropy`	0.266	0.319	0.778	+0.459
`flat_entropy_me`†	0.251	0.304	0.737	+0.433
`flat_entropy_opp`	0.259	0.308	0.746	+0.438

Table 2: The $\delta = 0.05$ arm against two opponents, head to head on the same checkpoint. Elo favours the $\delta = 0.05$ arm. ΔF is its mean state forgiveness minus its opponent's.

ϵ	score	Elo [95% CI]	ΔF
Opponent plays the π' argmax			
0.00	0.848	+297.9 [252.6, 353.5]	−0.020
0.10	0.675	+127.0 [81.6, 176.9]	−0.028
Opponent plays the $\hat{q}$ argmax ($\delta = 0$)			
0.00	0.497	− 1.7 [−34.4, 30.9]	+0.004
0.10	0.431	− 48.5 [−71.9, −25.5]	+0.007

Figure 1 shows the shape of the dependence, F is medium in the first few plies, falls to a minimum in the mid game, and rises steeply through the endgame. The trend is strong enough to explain the R^2, but the interquartile range at fixed material is wider than the span of the median curve across the mid game, so material fixes the level of F and leaves most of its variation unaccounted for. This is what the +0.43 margin over the trivial controls consists of.

The local and lookahead curves are indistinguishable along this axis, so the recursion adds nothing to the material dependence. Across all targets (Table 4) the local statistics correlate at 0.90 with each other but only 0.60–0.87 with the lookahead, which is therefore a genuinely different quantity, and that separation is carried entirely by variation at fixed material. Tree and flat aggregation correlate at 0.97-0.99, so the recursion adds little anywhere. The mover and opponent relative variants correlate at 0.92, but both are visit-weighted averages of the same local statistic over the same subtree at alternating plies, so this reflects the subtree average being a stable property of the position rather than the point of view being uninformative. The deployed target is therefore `flat_entropy_me`, which costs ≈ 0.04 in R^2 against the best available; §4.1 justifies the mover-only choice.

6.2 π' vs Q

Both arms are the same checkpoint with two selection rules, differing only in the loaded head, isolating the statistic. Since the head is evaluated after action is taken, the opposite statistic is used. The head is used for any position with actions within δ, since the actions are being evaluated, flat_entropy_opp is used, the opponent's version, since after the action the forgiveness is the opposite side.

After the tree search has finished and a move must be chosen, as laid out in § 3.2, there are two selection rules. The agent as trained and deployed plays the π' argmax, which retains the network prior weighted combined with search. The Q-argmax uses only the searched values once search is complete, discarding the prior.

The value term is bounded with $\sigma(\hat{q}) = (c_{visit} + \max_b N(b)) \cdot c_{scale} \cdot \hat{q}$. With values $c_{visit} = 50$, $\max_b N(b) = 191$ and $c_{scale} = 0.02$, the multiplier is $\sigma_{total} \approx 4.8$. Since the completed search values are rescaled to $[0, 1]$ before applying σ, the entire candidate set spans at most 4.8 from best to worst. The prior difference between a strongly preferred action and a plausible alternative action is of a similar order.

c_{scale} has to satisfy two requirements at the same time. The training target π' has to be soft, since a near one-hot target doesn't hold information about actions not chosen; the move selection wants the opposite, a low c_{scale} is what lets the prior overturn the search. Only the first was optimised for. This matters when evaluating because a δ selector takes the Q values within δ of the best and discards the priors, while a π' baseline does not. Setting $\delta = 0$ gives a greedy selector over the same values, removing the mismatch; every tiebreaker result therefore uses a $\delta = 0$ control.

Against a fixed clean benchmark the same mechanism gains +363.7 Elo over a π' control at $\epsilon = 0$. Reversing the head parity, which inverts the intended direction of the forgiveness, still gains +54 Elo (Table 8).

The parity check qualifies this rather than settling it, under perturbation, the two parities are indistinguishable at the resolution available here: +242 against +184 at $\epsilon = 0.05$ and +149 against +144 at $\epsilon = 0.10$, and +4 against +54.1 at $\epsilon = 0.15$, where the reversed head is ahead. They all fall within a per-arm interval of roughly ±45 Elo. Since the sign of the difference is not stable across ϵ, nothing distinguishes the intended direction of the statistic from its reverse. In clean play they are separated, and since the heads reach comparable fits to their targets, ($R^2 = 0.746, R^2 = 0.737$) the separation is unlikely to be a difference in the heads' quality. The margin under perturbation is consistent with the mismatch alone. The clean-play separation is not and suggests that selecting for the mover's own post-move slack correlates with move quality at this level of play, independently of any tolerance to error, an effect the robustness hypothesis does not predict and which the present experiments cannot separate from it.

6.3 δ tiebreaking

Testing the forgiveness using a δ tiebreak, where, at each search selection, the actions with a $\hat{q}$ value within δ of the top value are put into a set, and the action with the highest forgiveness value from these is chosen as the action. I've compared $\delta = 0$ as the benchmark to $\delta = 0.05$ to combat the selection difference mentioned in the above section. These runs are the same checkpoint, only differing in the selection of the near-optimal actions.

From the results in Table 6, with $\epsilon = 0$, clean play, no mistakes, the Elo difference is indistinguishable. With $\epsilon = 0.1$, making a mistake 10% of the time, the $\delta = 0$ arm wins by +48.5 Elo over 800 games, with a confidence interval that excludes 0; an earlier 400 game run at the same setting gave +40.1 $[7.5, 73.5]$, so the effect held when the sample was doubled. The mean forgiveness F is similar in both, but with an increase for the $\delta = 0.05$ arm which is the direction sought after. The switching happens 40% of the time, with a small value sacrifice, but with the forgiveness only increasing $0.004 - 0.007$, this change is too small to see a noticeable difference.

6.4 Shaping as a training objective

For the shaped run, with the value objective, two forks from the same checkpoint with the same config apart from adding a forgiveness shaping β. These runs were carried on for the same amount of iterations each (1825) and are benchmarked against the fork point.

Figure 2 plots both halves of the result, on the left, the shaped fork beats the baseline at every pertubation level, although the margin reduces. It is the stronger player but also the faster degrading one, which is the opposite of what the shaping was introduced for. The shaped fork sits below the baseline on $F_{me} - \gamma F_{opp}$, the quantity its objective maximises, at all four levels, and the intervals overlap throughout, so the run cannot be said to have moved its own objective in either direction. The most likely reading is the one in § 6.6: at $\beta = 0.03$, well below the 0.031 search noise, the bonus acts as a small systematic perturbation of the value target and regularises training rather than steering the search toward flat states.

6.5 MaxEnt at matched cost

The policy-entropy comparison of MaxEnt was partially tested at deployment, using the same checkpoint and search, the played move sampled as $a \sim \text{softmax}(\hat{q}(a)/\alpha)$ over the halving candidates. To make the compari-

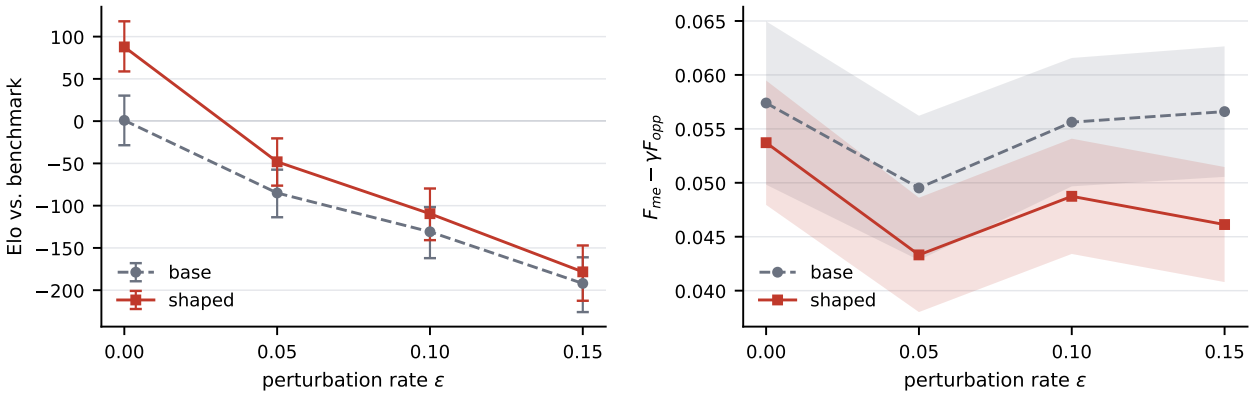


Figure 2: Trained value shaping, plotting Table 7. Left: Elo against the fork point; bars are 95% intervals on the mean per-game score. Right: the shaped objective $F_{me} - \gamma F_{opp}$ at $\gamma = 0.85$, formed per game against the benchmark; bands are 95% intervals. 400 games per arm per ϵ.

son fair, the two policies should pay the same price in expected value, so α was swept over $\{0.01, 0.025, 0.05\}$ and set to 0.025, whose sacrifice per move of 0.0044 matches the tiebreaks 0.0045.

Under benchmark playing, at $\epsilon = 0$ MaxEnt scored +166.2 Elo against the fixed opponent, while the baseline scored −50.7, and at $\epsilon = 0.1$ they scored −52.5 against −79.5 respectively against the benchmark. Both arms sit on opposite sides of the selection-rule effect of Section 6.2, so the clean-board margin reflects that difference rather than the entropy of the acting policy. At matched value cost, sampling proportional to $\exp(\hat{q}/\alpha)$ retained no more strength under perturbation rather than acting greedily on the same values.

6.6 Discussion

The main issue is the steering being too small, the tiebreaker only achieving 0.004 greater average forgiveness over the baseline, and the shaped run achieving lower forgiveness than the baseline. For the tiebreaker, the δ was calibrated to be conservative, so as not to displace meaningfully stronger actions, this was done to give some safety but didn't achieve the desired effect. For the shaping run, the forgiveness wasn't actually optimised, but the difference in forgiveness from the mover's point of view to the opponent's; therefore, it can be logical that the average forgiveness is lower if what's optimised is lowering the opponent's forgiveness.

The injected error is drawn from the halving finalists, so the replacement is the runner-up rather than an arbitrary legal move. This is the error model most favourable to the hypothesis; the mistake chosen was taken from the near-optimal set, and yet no benefit was observed.

Chess also may be a bad environment for these tests, since the value states with many near-optimal actions are also states where less is at stake. Figure 1 shows this directly. The most forgiving states are endgames, and the least forgiving are early to mid-games, once the opening book is left but before material comes off. The initial position scores highly, but F has already fallen by half by the time three or four pieces are traded, so the forgiving region is not the opening but the endgame, which is also where the fewest games are decided.

The two methods disagreed in the direction, tiebreaking was similar to not, but lost more Elo under perturbations, whereas the shaping had a 88 Elo gain without perturbations but closing the gap with perturbations applied. The bonus likely acted like a regulariser on the value target rather than steering towards flat states. The right panel of Figure 2 is the direct evidence for this: the shaped objective did not rise.

7 Conclusion

This work treated error tolerance as a property of the state, formalising forgiveness within a state, estimating it during search and targeting it at selection and through a training objective. I have established that the forgiveness statistic that I defined is a real, learnable quantity, with the forgiveness heads reaching $R^2 \approx 0.74$, with the lookahead variants measuring a genuinely different quantity from the local ones, while tree and flat

aggregation did not differ. The δ tiebreak steered as designed, although only slightly, switching 40% of moves and achieving a small average forgiveness increase.

Steering towards forgiveness, however, did not improve the robustness, with the tiebreak variant losing 48.5 Elo at $\epsilon = 0.1$ compared to its benchmark and with the shaped run gaining 88 Elo, but degrading faster with more perturbation and not moving to more forgiving states or moving $F_{me} - F_{opp}$ in the intended direction. Prior-regularised selection sits 300–360 Elo below search-greedy selection because one constant governs both target softness and acting sharpness, while the two require opposite values.

This is a negative result about chess rather than about forgiveness. The states this statistic defines as forgiving are endgames, which are also the states with least at stake. The environment couples the quantity being steered towards to something it should be separated from. The hypothesis is therefore untested rather than falsified; and remains as open in a setting with fixed dimension action space or one without forced moves.

Limitations: Realistically, the main failure from this paper came from the lack of strength of the trained models, when calibrated to a rough Elo strength, around 1200 was achieved by the baseline; however, this is not strong enough to have priors that are good enough to properly define the forgiveness. Since the value and Q values all come from search, these are too noisy without a strong enough model to reliably give forgiveness values that actually make a difference. A further experiment would be to take a much stronger model, which I couldn't train in the circumstances, and apply the forgiveness statistic to it. With better-defined Q values and closer-to-optimal priors, it's much more likely that the forgiveness would have an impact.

MiniZero also proposes progressive simulation, raising the search budget as training goes on to allocate computation more effectively (Wu et al., 2024). Since the quality of the Q-estimates are the biggest limitation, realocating budgets across training is a solid method to a stronger checkpoint that the forgiveness requires.

These results leave the following untested. **Candidate set width.** *stat_width* was fixed at four throughout and never varied. Entropy over four actions cannot see near-optimal moves ranked fifth or lower, so F understates the statistic in higher branching positions; the steering here is over a narrow set and not the true near-optimal one. **Undefined statistics.** The frequency where fewer than two actions qualify at a root is not measured; these rows are undefined and contribute nothing to loss or aggregates. This changes the real fraction of decisions tiebreak acts on. **Mis-execution model.** Replacements are drawn from the top 4 finalists, so the injected mistake is runner-up quality, and a uniform replacement amount of legal moves is untested. **MaxEnt comparison.** § 6.5 matches the two policies on per-move value sacrifice; this could be done on clean Elo instead. No agent is trained on this policy, and the test is only deployment time sampled. **Perturbation coverage.** The tiebreak was run at $\epsilon \in \{0, 0.1\}$ and the shaped fork at $\epsilon \in \{0, 0.05, 0.1, 0.15\}$, it can't be excluded that either could work better at higher ϵ. **Sibling estimators.** The $\hat{q}$'s are visit averages but don't have variance awareness or shrinkage correction. There are more techniques to give better Q estimates, given that estimate noise is the largest constraint.

Future work: Since the runs moved F by at most 0.004–0.007, the most direct next step is a definition that can move further. Exploring forgiveness over states revealed most states are either very forgiving or very brittle, mostly since there are many states with few move options or forced moves, which skew the statistic. Handling forced or narrow action spaces separately instead of excluding them can change things; they are legitimate moves in which blundering is impossible. A recursive halving schedule, as in H-MCTS, would also extend comparable sibling estimates past the root. Alternatively, this could be addressed in a change of environment, one that doesn't have such variable action spaces.

Continuous control is the natural candidate. Ayabe et al. inject random and adversarial perturbations into the joint torque signals of legged robots and find that offline methods degrade steeply under them (Ayabe et al., 2025), which is the same failure mode studied here in a setting where the action space has fixed dimension and no forced moves, so the normalisation problem described above does not arise and the statistic can be better compared across states. Although the statistic would need to be redefined there, since a Boltzmann distribution over a finite candidate set has no direct equivalence, and flatness would have to be measured as the volume of the near-optimal region instead.

References

Shingo Ayabe, Takuto Otomo, Hiroshi Kera, and Kazuhiko Kawamoto. Robustness evaluation of offline reinforcement learning for robot control against action perturbations. *International Journal of Advanced Robotic Systems*, 2025. doi: 10.1177/17298806251360454.

Igor Babuschkin, Kate Baumli, Alison Bell, Surya Bhupatiraju, Jake Bruce, Peter Buchlovsky, David Budden, Trevor Cai, Aidan Clark, Ivo Danihelka, Claudio Fantacci, Jonathan Godwin, Chris Jones, Tom Hennigan, Matteo Hessel, Steven Kapturowski, Thomas Keck, Iurii Kemaev, Michael King, Lena Martens, Vladimir Mikulik, Tamara Norman, John Quan, George Papamakarios, Roman Ring, Francisco Ruiz, Alvaro Sanchez, Rosalia Schneider, Eren Sezener, Stephen Spencer, Srivatsan Srinivasan, Wojciech Stokowiec, and Fabio Viola. The DeepMind JAX ecosystem. `https://github.com/google-deepmind/mctx`, 2021. `mctx`: Monte-Carlo tree search in JAX.

Stefan Behnel, Robert Bradshaw, Craig Citro, Lisandro Dalcin, Dag Sverre Seljebotn, and Kurt Smith. Cython: The best of both worlds. *Computing in Science & Engineering*, 13(2):31–39, 2011. doi: 10.1109/MCSE.2010.118.

Marc G. Bellemare, Georg Ostrovski, Arthur Guez, Philip S. Thomas, and Rémi Munos. Increasing the action gap: New operators for reinforcement learning. In *Proceedings of the 30th AAAI Conference on Artificial Intelligence*, pp. 1476–1483, 2016.

Tristan Cazenave. Sequential halving applied to trees. *IEEE Transactions on Computational Intelligence and AI in Games*, 7(1):102–105, 2015.

Ivo Danihelka, Arthur Guez, Julian Schrittwieser, and David Silver. Policy improvement by planning with Gumbel. In *International Conference on Learning Representations (ICLR)*, 2022.

Benjamin Eysenbach and Sergey Levine. Maximum entropy RL (provably) solves some robust RL problems. In *International Conference on Learning Representations (ICLR)*, 2022.

Amir-massoud Farahmand. Action-gap phenomenon in reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 24, pp. 172–180, 2011.

Pierre Foret, Ariel Kleiner, Hossein Mobahi, and Behnam Neyshabur. Sharpness-aware minimization for efficiently improving generalization. In *International Conference on Learning Representations (ICLR)*, 2021.

Keith Frankston and Benjamin Howard. Accelerating Monte-Carlo tree search with optimized posterior policies, 2026.

Jean-Bastien Grill, Florent Altché, Yunhao Tang, Thomas Hubert, Michal Valko, Ioannis Antonoglou, and Rémi Munos. Monte-Carlo tree search as regularized policy optimization. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*, volume 119 of *Proceedings of Machine Learning Research*, pp. 3769–3778. PMLR, 2020.

Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, volume 80 of *Proceedings of Machine Learning Research*, pp. 1861–1870. PMLR, 2018.

Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, Julian Taylor, Sebastian Berg, Nathaniel J. Smith, Robert Kern, Matti Picus, Stephan Hoyer, Marten H. van Kerkwijk, Matthew Brett, Allan Haldane, Jaime Fernández del Río, Mark Wiebe, Pearu Peterson, Pierre Gérard-Marchant, Kevin Sheppard, Tyler Reddy, Warren Weckesser, Hameer Abbasi, Christoph Gohlke, and Travis E. Oliphant. Array programming with NumPy. *Nature*, 585(7825):357–362, 2020. doi: 10.1038/s41586-020-2649-2.

Sepp Hochreiter and Jürgen Schmidhuber. Flat minima. *Neural Computation*, 9(1):1–42, 1997. doi: 10.1162/neco.1997.9.1.1.

Garud N. Iyengar. Robust dynamic programming. *Mathematics of Operations Research*, 30(2):257–280, 2005. doi: 10.1287/moor.1040.0129.

Zohar Karnin, Tomer Koren, and Oren Somekh. Almost optimal exploration in multi-armed bandits. In *Proceedings of the 30th International Conference on Machine Learning (ICML)*, volume 28 of *Proceedings of Machine Learning Research*, pp. 1238–1246. PMLR, 2013.

Nitish Shirish Keskar, Dheevatsa Mudigere, Jorge Nocedal, Mikhail Smelyanskiy, and Ping Tak Peter Tang. On large-batch training for deep learning: Generalization gap and sharp minima. In *International Conference on Learning Representations (ICLR)*, 2017.

Levente Kocsis and Csaba Szepesvári. Bandit based Monte-Carlo planning. In *Proceedings of the 17th European Conference on Machine Learning (ECML)*, pp. 282–293. Springer, 2006.

Andrew Y. Ng, Daishi Harada, and Stuart Russell. Policy invariance under reward transformations: Theory and application to reward shaping. In *Proceedings of the 16th International Conference on Machine Learning (ICML)*, pp. 278–287, 1999.

Arnab Nilim and Laurent El Ghaoui. Robust control of Markov decision processes with uncertain transition matrices. *Operations Research*, 53(5):780–798, 2005. doi: 10.1287/opre.1050.0216.

Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 32, pp. 8024–8035, 2019.

Tom Pepels, Tristan Cazenave, Mark H. M. Winands, and Marc Lanctot. Minimizing simple and cumulative regret in Monte-Carlo tree search. In *Computer Games Workshop (CGW) at ECAI 2014*, volume 504 of *Communications in Computer and Information Science*, pp. 1–15. Springer, 2014.

Lerrel Pinto, James Davidson, Rahul Sukthankar, and Abhinav Gupta. Robust adversarial reinforcement learning. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, volume 70 of *Proceedings of Machine Learning Research*, pp. 2817–2826. PMLR, 2017.

Aravind Rajeswaran, Sarvjeet Ghotra, Balaraman Ravindran, and Sergey Levine. EPOpt: Learning robust neural network policies using model ensembles. In *International Conference on Learning Representations (ICLR)*, 2017.

Christopher D. Rosin. Multi-armed bandits with episode context. *Annals of Mathematics and Artificial Intelligence*, 61(3):203–230, 2011. doi: 10.1007/s10472-011-9258-6.

Dominic Sagers, Mark H. M. Winands, and Dennis J. N. J. Soemers. Anytime sequential halving in Monte-Carlo tree search. In *Advances in Computer Games (ACG)*. Springer, 2025. doi: 10.1007/978-3-031-86585-5_8.

David Silver, Julian Schrittwieser, Karen Simonyan, Ioannis Antonoglou, Aja Huang, Arthur Guez, Thomas Hubert, Lucas Baker, Matthew Lai, Adrian Bolton, Yutian Chen, Timothy Lillicrap, Fan Hui, Laurent Sifre, George van den Driessche, Thore Graepel, and Demis Hassabis. Mastering the game of Go without human knowledge. *Nature*, 550(7676):354–359, 2017. doi: 10.1038/nature24270.

David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc Lanctot, Laurent Sifre, Dharshan Kumaran, Thore Graepel, Timothy Lillicrap, Karen Simonyan, and Demis Hassabis. A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play. *Science*, 362(6419):1140–1144, 2018. doi: 10.1126/science.aar6404.

Chen Tessler, Yonathan Efroni, and Shie Mannor. Action robust reinforcement learning and applications in continuous control. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*, volume 97 of *Proceedings of Machine Learning Research*, pp. 6215–6224. PMLR, 2019.

Josh Tobin, Rachel Fong, Alex Ray, Jonas Schneider, Wojciech Zaremba, and Pieter Abbeel. Domain randomization for transferring deep neural networks from simulation to the real world. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 23–30. IEEE, 2017. doi: 10.1109/IROS.2017.8202133.

David J. Wu. Accelerating self-play learning in Go, 2019.

Ti-Rong Wu, Hung Guei, Pei-Chiun Peng, Po-Wei Huang, Ting Han Wei, Chung-Chin Shih, and Yun-Jui Tsai. MiniZero: Comparative analysis of AlphaZero and MuZero on Go, Othello, and Atari games. *IEEE Transactions on Games*, 2024.

Huan Zhang, Hongge Chen, Chaowei Xiao, Bo Li, Mingyan Liu, Duane Boning, and Cho-Jui Hsieh. Robust deep reinforcement learning against adversarial perturbations on state observations. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, 2020.

Brian D. Ziebart, Andrew Maas, J. Andrew Bagnell, and Anind K. Dey. Maximum entropy inverse reinforcement learning. In *Proceedings of the 23rd AAAI Conference on Artificial Intelligence*, pp. 1433–1438, 2008.

A Compute and reproducibility

Training ran on three NVIDIA Tesla V100 (32 GB) GPUs of a ten-GPU node with two Intel Xeon Gold 6248 CPUs and 512 GB of RAM. The deployed checkpoint is iteration 11275 of a single run: 113.7 hours of wall-clock, 61.9 seconds per iteration before a restart at iteration 2676 and 28.1 seconds after, and roughly 46 million self-play rows through a 600000 row buffer that saturates at iteration 146. The forgiveness heads train offline on 512 games from the frozen checkpoint, held out by contiguous tail so that rows sharing a game and a search cannot straddle the split. The shaped forks branch at iteration 11275 and run 1825 further iterations each.

B σ instantiation

The specific instantiation in the paper is $\sigma(\hat{q}(a)) = (c_{visit} + max_b N(b)) \cdot c_{scale} \cdot \hat{q}(a)$, which weights towards Q. σ grows with the visit count, so the balance between terms is set by how much of the budget has been spent, with a few visits σ is small and the target stays close to prior, while with many visits, the $\hat{q}$ term dominates and the prior contributes much less. This form multiplies the $\hat{q}$ values, meaning the sharpness of the target is dependent on how far apart the values are. The values are min-max rescaled onto $[0, 1]$ before σ is applied, which fixes the spread at 1 and leaves c_{scale} as the only thing controlling sharpness. Unvisited actions have no Q and are completed with v_{mix}, an interpolation between the root's network value and the prior-weighted mean of the visited actions, so the target covers every legal move rather than only the survivors.

C Elo

Elo differences invert $E = 1/(1 + 10^{-\Delta/400})$ on the mean per-game score, so $\Delta = -400 \log_{10}(1/E - 1)$. Zero is an even score and positive values favour the first-named arm; in head-to-head pairings the two arms' figures are equal and opposite by construction. Confidence bounds are 95% intervals on the mean per-game score transformed through the same expression. Absolute Elo is anchored against fixed agents at roughly 1000 and 1500, plus random selection and a one-step material agent.

D Optimisation

Optimisation uses AdamW at 4×10^{-4} with weight decay 10^{-4} on the trunk, policy and value parameters, and a second AdamW on the forgiveness head at 10^{-3} cosine-decayed to 3×10^{-4}. Training runs in mixed

precision under a persistent GradScaler. The head's optimiser is stepped only on batches containing at least one forgiveness-labelled row, since weight decay and stale momentum would otherwise shrink the head on every masked-out batch.

E First-play urgency

Plain PUCT assigns $Q = 0$ to unvisited children, which is a hidden bias rather than a neutral default: when the chooser is losing, every unexplored move scores above the moves already searched, and simulations are split across refuted alternatives. An unvisited child of s is instead assumed to be slightly worse than the moves already explored from s,

$$Q_{\mathrm{FPU}}(s) = \bar{Q}(s) - \texttt{fpu_reduction},$$

with `fpu_reduction` $= 0.25$ throughout. The score in §3.1 is evaluated with $Q(s,a) \leftarrow Q_{\mathrm{FPU}}(s)$ for any a with $N(s,a) = 0$, and setting the reduction to 0 recovers the legacy behaviour.

The reference value $\bar{Q}(s)$ differs by node type. Interior nodes accumulate value from the point of view of the player who moved *into* them, so the chooser-POV average is the negation of the running mean, $\bar{Q}(s) = -V(s)/N(s)$. The root has no such mover and its accumulator goes stale under subtree reuse, so it falls back to a visit-weighted mean over its children, whose values are already chooser POV

Table 3: Hyperparameters.

Parameter	Description	Value
Network and optimisation		
`channels`	Residual trunk width	128
`num_blocks`	Residual blocks	8
Input planes	Position encoding, 8×8	19
Action space	64×73	4672
`lr`	Learning rate (trunk, policy, value)	4×10^{-4}
`batch_size`	Rows per gradient step	256
`train_batches`	Gradient steps per iteration	64
Self-play		
`games_per_iter`	Games per iteration	192
`max_plies`	Ply cap before adjudication	300
`temp_moves`	Plies sampled at temperature 1	20
`full_search_prob`	Plies given the full budget	0.5
`fast_search_iterations`	Budget on capped plies	100
`buffer_capacity`	Replay buffer rows	600 000
Search		
`search_iterations`	Simulations per full-budget ply	800
`fpu_reduction`	First-play-urgency reduction	0.25
`value_target_lambda`	Weight on the game outcome z	0.7
`sh_m`	Root actions sampled, top-m	16
`stat_width`	Actions retained for statistics	4
`sh_c_visit`	c_{visit} in σ	50.0
`sh_c_scale`	c_{scale} in σ	0.02
Forgiveness		
`forgiveness_tau`	Boltzmann temperature τ	0.0178
`forgiveness_gamma`	Lookahead decay γ	0.85
`forgiveness_target_mode`	Statistic and aggregation	`flat_entropy_me`
`forgiveness_lr`	Head learning rate	1×10^{-3}
δ		0.05
β		0.03

Table 4: Pearson correlation between target definitions.

Target	(1)	(2)	(3)	(4)	(5)	(6)	(7)	(8)	(9)	(10)
Local only										
(1) `gap`	1.00									
(2) `entropy`	0.90	1.00								
Lookahead, both sides										
(3) `tree_gap`	0.72	0.79	1.00							
(4) `tree_entropy`	0.67	0.79	0.98	1.00						
(5) `flat_gap`	0.76	0.82	0.99	0.97	1.00					
(6) `flat_entropy`	0.70	0.83	0.97	0.99	0.98	1.00				
Lookahead, single point of view										
(7) `tree_gap_me`	0.71	0.78	0.99	0.97	0.97	0.96	1.00			
(8) `tree_gap_opp`	0.60	0.69	0.98	0.97	0.95	0.94	0.96	1.00		
(9) `flat_entropy_me`	0.75	0.87	0.95	0.97	0.96	0.98	0.95	0.90	1.00	
(10) `flat_entropy_opp`	0.61	0.73	0.95	0.97	0.95	0.97	0.92	0.95	0.92	1.00

Table 5: Trivial-feature controls for the forgiveness heads. Each control column is an ordinary least squares fit predicting the target from features read directly off the encoded planes, added cumulatively left to right; no network and no search are involved. The head is the trained head's R^2 on the same rows.

	Control R^2					
Target	pieces	+halfmv	+mat.	+per-type	**Head**	**Margin**
Local only						
`gap`	0.096	0.108	0.110	0.141	0.485	+0.344
`entropy`	0.133	0.146	0.146	0.190	0.602	+0.412
Lookahead, both sides						
`tree_gap`	0.280	0.282	0.284	0.322	0.762	+0.440
`tree_entropy`	0.299	0.301	0.304	0.344	0.767	+0.423
`flat_gap`	0.260	0.264	0.265	0.307	0.770	+0.463
`flat_entropy`	0.266	0.270	0.272	0.319	0.778	+0.459
Lookahead, single point of view						
`tree_gap_me`	0.292	0.293	0.294	0.329	0.747	+0.418
`tree_gap_opp`	0.303	0.303	0.310	0.342	0.753	+0.411
`flat_entropy_me`	0.251	0.256	0.258	0.304	0.737	+0.433
`flat_entropy_opp`	0.259	0.262	0.264	0.308	0.746	+0.438

Table 6: Forgiving tree using `flat_entropy_me`: choosing the best action every time ($\delta = 0$) against switching to the most forgiving action within $\delta = 0.05$. Elo is reported for the $\delta = 0$ arm. The clean row is 400 games, the perturbed row 800.

	Score				**Mean F**		**$\delta = 0.05$ arm**	
ϵ	$\delta = 0$	$\delta = 0.05$	**Elo**		$\delta = 0$	$\delta = 0.05$	switch	sacrifice
0.00	0.502	0.497	+1.7	[−30.9, 34.4]	0.5309	0.5349	0.395	0.0045
0.10	0.569	0.431	+48.5	[25.5, 71.9]	0.4985	0.5052	0.402	0.0046

Table 7: Trained value shaping. Two forks of one checkpoint, identical but for the shaping β, run for 1825 further iterations and each benchmarked against the fork point. The final pair of columns reports the quantity the shaped objective maximises, $F_{me} - \gamma F_{opp}$ at $\gamma = 0.85$, formed per game by pairing each arm's mean state forgiveness against the benchmark's in the same game; mean F alone does not describe what the alternating-sign backup optimises. 400 games per arm per ϵ.

	Score		**Elo vs. benchmark**		$F_{me} - \gamma F_{opp}$	
ϵ	base	shaped	base	shaped	base	shaped
0.00	0.501	0.624	+ 0.9	+ 87.8	0.0574	0.0537
0.05	0.380	0.431	− 85.0	− 48.1	0.0495	0.0433
0.10	0.320	0.347	−130.9	−109.5	0.0556	0.0487
0.15	0.249	0.264	−192.0	−178.3	0.0566	0.0461

Table 8: Parity check, both arms play a fixed third checkpoint, never perturbed. Δ is the forgiveness arm's Elo minus its own π' control. 200 games per arm per ϵ.

	Elo vs. benchmark			**Mean F**	
ϵ	π'	forg.	Δ	π'	forg.
Correct parity, `flat_entropy_opp`					
0.00	−130.9	+232.8	+363.7	0.333	0.454
0.05	−132.9	+109.5	+242.4	0.359	0.439
0.10	−168.4	− 19.1	+149.3	0.405	0.452
0.15	−137.0	−132.9	+ 4.1	0.395	0.444
Reversed parity, `flat_entropy_me`					
0.00	−139.0	− 85.0	+ 54.0	0.327	0.380
0.05	−135.0	+ 49.0	+184.0	0.359	0.385
0.10	−159.8	− 15.6	+144.2	0.403	0.405
0.15	−141.0	− 86.9	+ 54.1	0.391	0.412